

Talkify By Kaysan Shaikh

Talkify is a real-time application built using the MERN stack. It provides users with a seamless and interactive chatting experience.

Features: -

- **Real-time Messaging:** Users can chat with each other in real time, sending and receiving messages instantly without the need to refresh the page.
- **User Authentication:** The application supports user registration and authentication, ensuring secure access to the chat functionality.
- **Group Chats:** Users can create or join group chats, allowing them to communicate with multiple participants simultaneously.
- **Message History:** The application keeps a history of previous messages, allowing users to view their chat history and scroll through past conversations.
- **Emojis and Media Sharing:** Users can enhance their conversations by sending emojis and sharing media files such as images or videos.

Technologies Used: -

Talkify is built using the MERN stack, which consists of the following technologies:

- **MongoDB:** A NoSQL database used to store and manage user information, chat messages, and other relevant data.
- **Express.js:** A web application framework for Node.js that handles server-side logic and API routes.
- **React:** A JavaScript library for building user interfaces, used to create the client-side of the application.
- **Node.js:** A JavaScript runtime environment used for server-side development and running the Express.js server.
- **Socket.IO:** A library that enables real-time, bidirectional communication between the server and the clients, facilitating instant messaging.
- **SCSS:** A CSS preprocessor that provides additional features and enhancements to standard CSS, such as variables, nesting, and modular organization.

Steps: -

To use the Talkify chat application, follow these steps:

1. Clone the Repository:

```
git clone https://github.com/Kaysanshaikh/FutureIntern_FSD_04
```

2. Navigate to the Project Directory:

```
cd FutureIntern_FSD_04
```

3. Install the Dependencies (For Server):

```
cd server  
npm install
```

4. Install the Dependencies (For Client):

```
cd public  
npm install
```

5. Set Up the Environment Variables:

- Create a .env file in the root directory.
- Specify the required environment variables, such as:
 - MONGODB_URI for the MongoDB connection URI.
 - JWT_SECRET for the JWT secret key.
 - Any other necessary configurations.

6. Start the Development Server:

```
npm start
```

7. Access the Application:

- Open your web browser and go to <http://localhost:3000> to access the application.
-

The image shows the MongoDB Compass application window. On the left is a dark green sidebar with the 'Compass' logo and a settings gear icon. Below the logo is a 'New connection' button with a plus icon. Further down are two sections: 'Saved connections' and 'Recents'. 'Saved connections' lists 'Kaysan Shaikh' with a timestamp 'Aug 27, 2024, 8:51 PM'. 'Recents' lists 'localhost:27017' with a timestamp 'Aug 27, 2024, 8:50 PM'. The main area is titled 'New Connection' and contains the text 'Connect to a MongoDB deployment'. Below this is a 'URI' field with the value 'mongodb://localhost:27017/'. To the right of the URI field is a toggle switch for 'Edit Connection String' which is currently turned on. Below the URI field is a link for 'Advanced Connection Options'. At the bottom of the main area are three buttons: 'Save', 'Save & Connect', and 'Connect'. On the right side of the window, there is a light blue sidebar with text: 'New to Compass and don't have a cluster?', 'If you don't already have a cluster, you can create one for free using MongoDB Atlas', a 'CREATE FREE CLUSTER' button, 'How do I find my connection string in Atlas?', 'If you have an Atlas cluster, go to the Cluster view. Click the "Connect" button for the cluster to which you wish to connect.', 'See example', 'How do I format my connection string?', and 'See example'.

MongoDB Compass

localhost:27017/temp.chats

Connect Edit View Collection Help

localhost:27017

...

My Queries

Performance

Databases

Search

admin

blink

config

kaysan

local

temp

chats

messages

records

users

test

chats

+

localhost:27017

>

temp

>

chats

Documents 5

Aggregations

Schema

Indexes 1

Validation

Type a query: { field: 'value' } or [Generate query](#)

Explain

Reset

Find

Options

ADD DATA

EXPORT DATA

UPDATE

DELETE

1 - 5 of 5

←

→

≡

{}

□

_id: ObjectId('66cdebc18251a6f00bdfc149')

isGroupChat: true

chatName: "Aniverse"

users: Array (1)

groupAdmin: ObjectId('66cdebc358251a6f00bdfc107')

groupPic: "1724776577651_Marvel.jpg"

createdAt: 2024-08-27T15:07:45.067+00:00

updatedAt: 2024-08-27T16:36:17.913+00:00

__v: 0

latestMessage: ObjectId('66cdebc8a251a6f00bdfc151')

_id: ObjectId('66cdecfb8251a6f00bdfc1bd')

isGroupChat: true

chatName: "Buddy"

users: Array (3)

groupAdmin: ObjectId('66cdec808251a6f00bdfc170')

groupPic: "1724776619109_h.jpg"

createdAt: 2024-08-27T15:11:59.464+00:00

updatedAt: 2024-08-27T16:37:10.856+00:00

__v: 0

latestMessage: ObjectId('66ce00b61c54167a5313fabc')

_id: ObjectId('66cdecfba251a6f00bdfc1d8')

isGroupChat: true

chatName: "Buddy"

users: Array (1)

> MONGOSH

MongoDB Compass - localhost:27017/temp.messages

Connect Edit View Collection Help

localhost:27017

My Queries Performance Databases Search

admin blink config kaysan local temp chats messages records users test

messages

localhost:27017 > temp > messages

Documents Aggregations Schema Indexes 1 Validation

Type a query: { field: 'value' } or [Generate query](#)

Explain Reset Find Options

ADD DATA EXPORT DATA UPDATE DELETE

1 - 11 of 11

```
{
  "_id": ObjectId("66cdeba8251a6f00bdf151"),
  "content": "Heyyyyyy",
  "attachment": "",
  "sender": ObjectId("66cdeb358251a6f00bdf107"),
  "chat": ObjectId("66cdeb18251a6f00bdf149"),
  "createdAt": 2024-08-27T15:07:54.673+00:00,
  "updatedAt": 2024-08-27T15:07:54.673+00:00,
  "__v": 0
}
```

```
{
  "_id": ObjectId("66cdecc8251a6f00bdf1c5"),
  "content": "hiiiiiiiiii",
  "attachment": "",
  "sender": ObjectId("66cdec808251a6f00bdf170"),
  "chat": ObjectId("66cdecfb8251a6f00bdf1bd"),
  "createdAt": 2024-08-27T15:12:13.750+00:00,
  "updatedAt": 2024-08-27T15:12:13.750+00:00,
  "__v": 0
}
```

```
{
  "_id": ObjectId("66cdec8251a6f00bdf1e0"),
  "content": "hi",
  "attachment": "",
  "sender": ObjectId("66cdec808251a6f00bdf170"),
  "chat": ObjectId("66cdecfb8251a6f00bdf1bd"),
  "createdAt": 2024-08-27T15:13:06.873+00:00,
  "updatedAt": 2024-08-27T15:13:06.873+00:00,
  "__v": 0
}
```

> MONGOSH

MongoDB Compass - localhost:27017/temp.users

Connect Edit View Collection Help

localhost:27017

My Queries Performance Databases Search

admin blink config kaysan local temp chats messages records users test

users

localhost:27017 > temp > users

Documents 5 Aggregations Schema Indexes 3 Validation

Type a query: { field: 'value' } or [Generate query](#)

Explain Reset Find Options

ADD DATA EXPORT DATA UPDATE DELETE

1 - 5 of 5

```
{
  "_id": ObjectId("66cdeac18251a6f00bdf0ff"),
  "username": "Kaysan Shaikh",
  "email": "shaikhkaysan@gmail.com",
  "password": "$2b$10$1k04bo2T.E9.VA4eflASu5zUTM1DfbelwqzvKg4l2CI8k1wL1f06",
  "profilePic": "1724776559080_Photo.jpg",
  "isAdmin": false,
  "__v": 0
}
```

```
{
  "_id": ObjectId("66cdeb358251a6f00bdf107"),
  "username": "Prathamesh Vaidya",
  "email": "prathameshvaidyachomu@gmail.com",
  "password": "$2b$10$ah9x7eSP/ILjR..lFAs6OeWfjHgC3sQcQEm22Jn5nNPN2vWqQMsze",
  "profilePic": "1724771237502_8.jpg",
  "isAdmin": false,
  "__v": 0
}
```

```
{
  "_id": ObjectId("66cdec808251a6f00bdf170"),
  "username": "Kaysan",
  "email": "kaysan@gmail.com",
  "password": "$2b$10$8wGTVgOLW5L8yR6tCKam010nC3XyE3b8H1Gb2XyOtqPcxTQKjhAW",
  "profilePic": "1724771456829_7.jpg",
  "isAdmin": false,
  "__v": 0
}
```

```
{
  "_id": ObjectId("66cdec7c8251a6f00bdf268")
}
```

> MONGOSH

Code: -

```
File Edit Selection View Go Run Terminal Help
Talkify by Kaysan Shaikh

EXPLORER
  OPEN EDITORS
    Login.jsx public
  TALKIFY...
  > vscode
  > node_modules
  > public
  > node_modules
  > public
  > src
  > assets
  > components
  > auth
  > authorization
    Login.jsx
    Register.jsx
  > Group
    CreateGroupCh...
    UpdateGroupCh...
    ChatContainer.jsx
    ChatInput.jsx
    Contacts.jsx
    Search.jsx
    SingleChat.jsx
    UpdateProfile.jsx
    UserInfo.jsx
  > config
  > context
  > pages
  > scss
  > utils
  > App.js
  > index.js
  > env
  > .gitignore
  > package-lock.json
  > package.json
  > README.md
  > server
  > config
  > generateToken.js
  > controller
  > OUTLINE
  > TIMELINE

Login.jsx
1 import { useState } from "react";
2 import { useNavigate } from "react-router-dom";
3 import "react-toastify/dist/ReactToastify.css";
4 import { ToastContainer, toast } from "react-toastify";
5
6 import { RxPerson, RxEyeOpen, RxEyeClosed } from "react-icons/rx";
7 import { FiLock } from "react-icons/fi";
8 import axios from "axios";
9 import { loginRoute } from "../utils/APIRoutes";
10 // import { toastOptions } from "../utils/constants";
11 import { toastOptions } from "C:/Users/Admin/Desktop/FutureIntern/Talkify by Kaysan Shaikh/public/src/utils/constants";
12
13 function Login({ isInactive }) {
14   const navigate = useNavigate();
15   const [values, setValues] = useState({ username: "", password: "" });
16   const [show, setShow] = useState(false);
17
18   //setting values of inputs
19   const handleChange = (event) => {
20     setValues({ ...values, [event.target.name]: event.target.value });
21   };
22
23   //checking if the entered data exists in the database
24   const validateForm = () => {
25     const { username, password } = values;
26     //checking if usernames match
27     if (username === "") {
28       toast.error("Username or Password is incorrect", toastOptions);
29       return false;
30     }
31     //checking if passwords match
32     else if (password === "") {
33       toast.error("Username or Password is incorrect", toastOptions);
34       return false;
35     }
36     return true;
37   };
38
39   const handleSubmit = async (event) => {
40     event.preventDefault();
41     if (validateForm()) {
42       try {
43         const config = {
44           headers: {
45             "Content-type": "application/json",
46           },
47         };
48         const { username, password } = values;
49         const { data } = await axios.post(loginRoute, {
50           username,
51           password,
52         });
53       } catch (error) {
54         //
55       }
56     }
57   };
58 }
59
60 export default Login;
```

```
File Edit Selection View Go Run Terminal Help
Talkify by Kaysan Shaikh

EXPLORER
  OPEN EDITORS
    Register.jsx public
  TALKIFY...
  > vscode
  > node_modules
  > public
  > node_modules
  > public
  > src
  > assets
  > components
  > auth
  > authorization
    Login.jsx
    Register.jsx
  > Group
    CreateGroupCh...
    UpdateGroupCh...
    ChatContainer.jsx
    ChatInput.jsx
    Contacts.jsx
    Search.jsx
    SingleChat.jsx
    UpdateProfile.jsx
    UserInfo.jsx
  > config
  > context
  > pages
  > scss
  > utils
  > App.js
  > index.js
  > env
  > .gitignore
  > package-lock.json
  > package.json
  > README.md
  > server
  > config
  > generateToken.js
  > controller
  > OUTLINE
  > TIMELINE

Register.jsx
1 import { useState } from "react";
2 import axios from "axios";
3 import { useNavigate } from "react-router-dom";
4 import { RxPerson, RxEyeOpen, RxEyeClosed } from "react-icons/rx";
5 import { RiOutlineAddPhoto } from "react-icons/ri";
6 import { FiMail, FiLock, FiUnlock } from "react-icons/fi";
7 import { ToastContainer, toast } from "react-toastify";
8 import "react-toastify/dist/ReactToastify.css";
9 import { registerRoute } from "../utils/APIRoutes";
10 // import { toastOptions } from "../utils/constants";
11 import { toastOptions } from "C:/Users/Admin/Desktop/FutureIntern/Talkify by Kaysan Shaikh/public/src/utils/constants";
12
13 function Register({ isInactive }) {
14   const navigate = useNavigate();
15   const [show, setShow] = useState(false);
16   const [showConfirm, setShowConfirm] = useState(false);
17   const [isActive, setIsActive] = useState("not-active");
18
19   //creating useState hook and setting empty values into it
20   const [values, setValues] = useState({
21     username: "",
22     email: "",
23     password: "",
24     confirmPassword: "",
25     profilePic: "",
26   });
27
28   //setting values of inputs
29   const handleChange = (event) => {
30     setValues({ ...values, [event.target.name]: event.target.value });
31   };
32
33   //setting an image value
34   const imageUpload = (event) => {
35     setValues({ ...values, profilePic: event.target.files[0] });
36     setIsActive("active");
37   };
38
39   const handleValidation = () => {
40     const { password, confirmPassword, username, email } = values;
41     if (password !== confirmPassword) {
42       //throwing an error
43       toast.error("Password and confirm password should be same.", toastOptions);
44       return false;
45     }
46     //if username is shorter than 3 characters
47     if (username.length < 3) {
48       //
49     }
50   };
51 }
52
53 export default Register;
```

```
File Edit Selection View Go Run Terminal Help
Talkify by Kaysan Shaikh

EXPLORER
src
  components
    Group
      CreateGroupChat.jsx
      UpdateGroupChat.jsx
      ChatContainer.jsx
      ChatInput.jsx
      Contacts.jsx
      Search.jsx
      SingleChat.jsx
      UpdateProfile.jsx
      UserInfo.jsx
  config
  context
  pages
  scss
  utils
  App.js
  index.js
  env
  .gitignore
  package-lock.json
  README.md
  yarn.lock
  server
  config
  generateToken.js
  controller
  OUTLINE
  TIMELINE

CreateGroupChat.jsx
1 import { useState } from "react";
2 import { ToastContainer, toast } from "react-toastify";
3 import { GrFormClose } from "react-icons/gn";
4 import { BsPencil } from "react-icons/bs";
5 import { AIOutlineUserAdd } from "react-icons/ai";
6
7 import axios from "axios";
8
9 import { ChatState } from "../../context/ChatProvider";
10 import { groupChatRoute, allUsersRoute } from "../../utils/APIRoutes";
11 // import { toastOptions } from "../../utils/constants";
12 import { toastOptions } from "C:/Users/Admin/Desktop/FutureIntern/Talkify by Kaysan Shaikh/public/src/utils/constants";
13
14 import UserListItem from "../../aauz/UserListItem";
15 import UserBage from "../../aauz/UserBage";
16
17 function GroupChatCreate({ setModalActive }) {
18   const [groupName, setGroupName] = useState("");
19   const [selectedUsers, setSelectedUsers] = useState([]);
20   const [search, setSearch] = useState("");
21   const [searchResult, setSearchResult] = useState([]);
22   const [loading, setLoading] = useState(false);
23   const [activeSearch, setActiveSearch] = useState(false);
24   const { user, chats, setChats } = ChatState();
25
26   const handleGroup = (userToAdd) => {
27     if (selectedUsers.includes(userToAdd)) {
28       toast.warning("User already added", toastOptions);
29       return;
30     } else {
31       setSelectedUsers([...selectedUsers, userToAdd]);
32     }
33   };
34
35   const handleSearch = async (query) => {
36     setSearch(query);
37     if (!query) {
38       return;
39     }
40     try {
41       setActiveSearch(true);
42       setLoading(true);
43       const config = {
44         headers: {
45           Authorization: `Bearer ${user.token}`,
46         },
47       };
48       const getUsers = async () => {
49         const { data } = await axios.get(`${allUsersRoute}?search=${query}`, config);
50         return data;
51       };
52     } catch (error) {
53       console.log(error);
54     }
55   };
56
57   const handleAddUser = async (user) => {
58     const chatId = selectedChat_id;
59     const userId = user_id;
60     const config = {
61       headers: {
62         Authorization: `Bearer ${user.token}`,
63       },
64     };
65     const selectedChat = { chatId, userId };
66     setSelectedChat(selectedChat);
67     setFetchAgain(fetchAgain);
68     setLoading(false);
69     setActiveSearch(false);
70     toast.success("User is successfully added to the group", toastOptions);
71     catch (error) {
72       toast.error(error.response.data.message, toastOptions);
73       setLoading(false);
74     }
75     setGroupName("");
76   };
77
78   const handleRemove = async (user) => {
79     if (selectedChat.groupAdmin_id !== user_id && user_id !== user_id) {
80       toast.error("Only admins can remove users from group chat", toastOptions);
81       return;
82     }
83     try {
84       setLoading(true);
85       const config = { headers: { Authorization: `Bearer ${user.token}` } };
86       const { data } = await axios.put(`${removeGroupChatRoute}`, { chatId: selectedChat_id, userId: user_id }, config);
87       user_id === user_id ? setSelectedChat() : setSelectedChat(data);
88       setFetchAgain(fetchAgain);
89       setLoading(false);
90     } catch (error) {
91       toast.error(error.response.data.message, toastOptions);
92       setLoading(false);
93     }
94     setGroupName("");
95   };
96
97   return (
98     <div className="update-group modal-wrapper">
99       <div className="modal-container $(activeSearch ? 'active-search' : '')">
100         <div className="close-button-wrapper">
101           <button className="close-button" onClick={() => { setModalActive('not') }} />
102         </div>
103         <div>
104           <div className="modal-header">
105             <div className="avatar">

```

```
File Edit Selection View Go Run Terminal Help
Talkify by Kaysan Shaikh

EXPLORER
src
  components
    Group
      CreateGroupChat.jsx
      UpdateGroupChat.jsx
      ChatContainer.jsx
      ChatInput.jsx
      Contacts.jsx
      Search.jsx
      SingleChat.jsx
      UpdateProfile.jsx
      UserInfo.jsx
  config
  context
  pages
  scss
  utils
  App.js
  index.js
  env
  .gitignore
  package-lock.json
  README.md
  yarn.lock
  server
  config
  generateToken.js
  controller
  OUTLINE
  TIMELINE

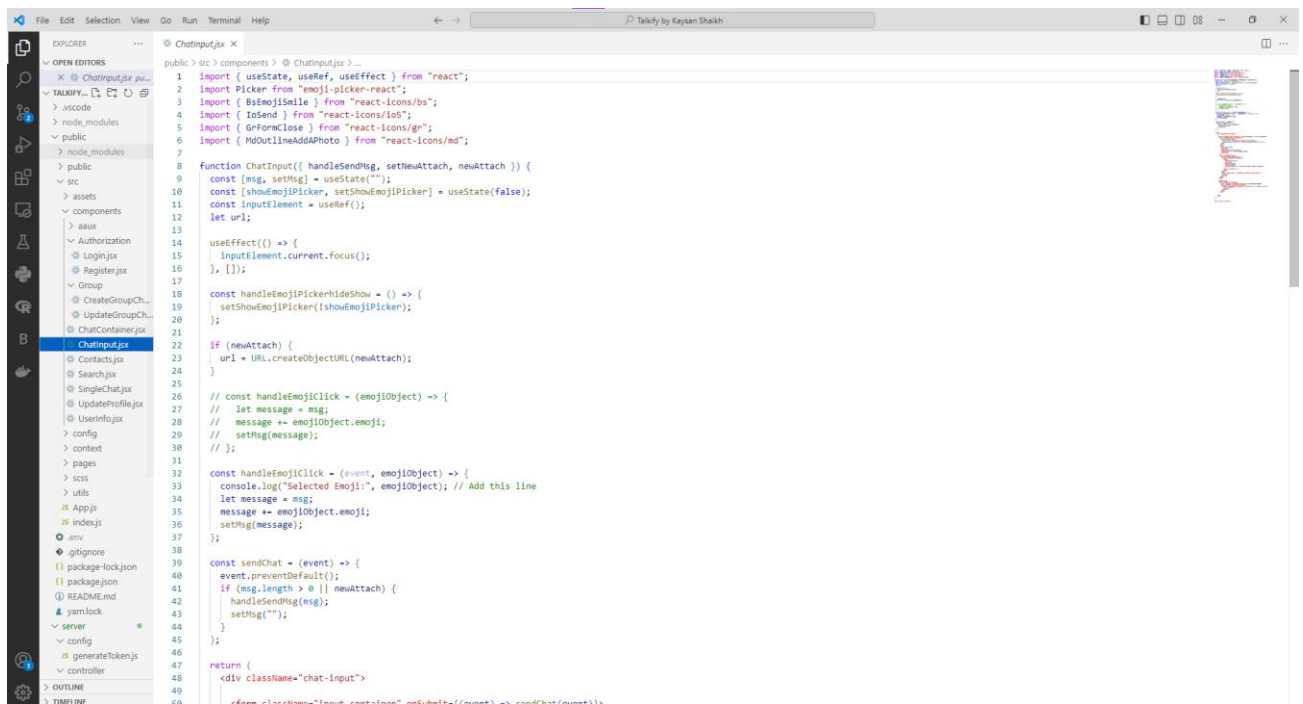
UpdateGroupChat.jsx
23 const UpdateGroupChat = ({ fetchAgain, setFetchAgain, setModalActive }) => {
24   const handleAddUser = async (user) => {
25     const chatId = selectedChat_id;
26     const userId = user_id;
27     const config = {
28       headers: {
29         Authorization: `Bearer ${user.token}`,
30       },
31     };
32     const selectedChat = { chatId, userId };
33     setSelectedChat(selectedChat);
34     setFetchAgain(fetchAgain);
35     setLoading(false);
36     setActiveSearch(false);
37     toast.success("User is successfully added to the group", toastOptions);
38     catch (error) {
39       toast.error(error.response.data.message, toastOptions);
40       setLoading(false);
41     }
42     setGroupName("");
43   };
44
45   const handleRemove = async (user) => {
46     if (selectedChat.groupAdmin_id !== user_id && user_id !== user_id) {
47       toast.error("Only admins can remove users from group chat", toastOptions);
48       return;
49     }
50     try {
51       setLoading(true);
52       const config = { headers: { Authorization: `Bearer ${user.token}` } };
53       const { data } = await axios.put(`${removeGroupChatRoute}`, { chatId: selectedChat_id, userId: user_id }, config);
54       user_id === user_id ? setSelectedChat() : setSelectedChat(data);
55       setFetchAgain(fetchAgain);
56       setLoading(false);
57     } catch (error) {
58       toast.error(error.response.data.message, toastOptions);
59       setLoading(false);
60     }
61     setGroupName("");
62   };
63
64   return (
65     <div className="update-group modal-wrapper">
66       <div className="modal-container $(activeSearch ? 'active-search' : '')">
67         <div className="close-button-wrapper">
68           <button className="close-button" onClick={() => { setModalActive('not') }} />
69         </div>
70         <div>
71           <div className="modal-header">
72             <div className="avatar">

```

```

ChatContainer.jsx X
public > src > components > ChatContainer.jsx > ...
151     setModalActive={setModalSubmitActive}
152     warnText={warnText}
153     submText={submText}
154     handleFunction={handleLeave}
155   />
156   : <></>
157 }
158 {modalSubmitActive === "delete"
159   ? <SubmitModal
160     setModalActive={setModalSubmitActive}
161     warnText={warnText}
162     submText={submText}
163     handleFunction={handleDelete}
164   />
165   : <></>
166 }
167 </div >
168 );
169 }
170
171 export default ChatContainer;

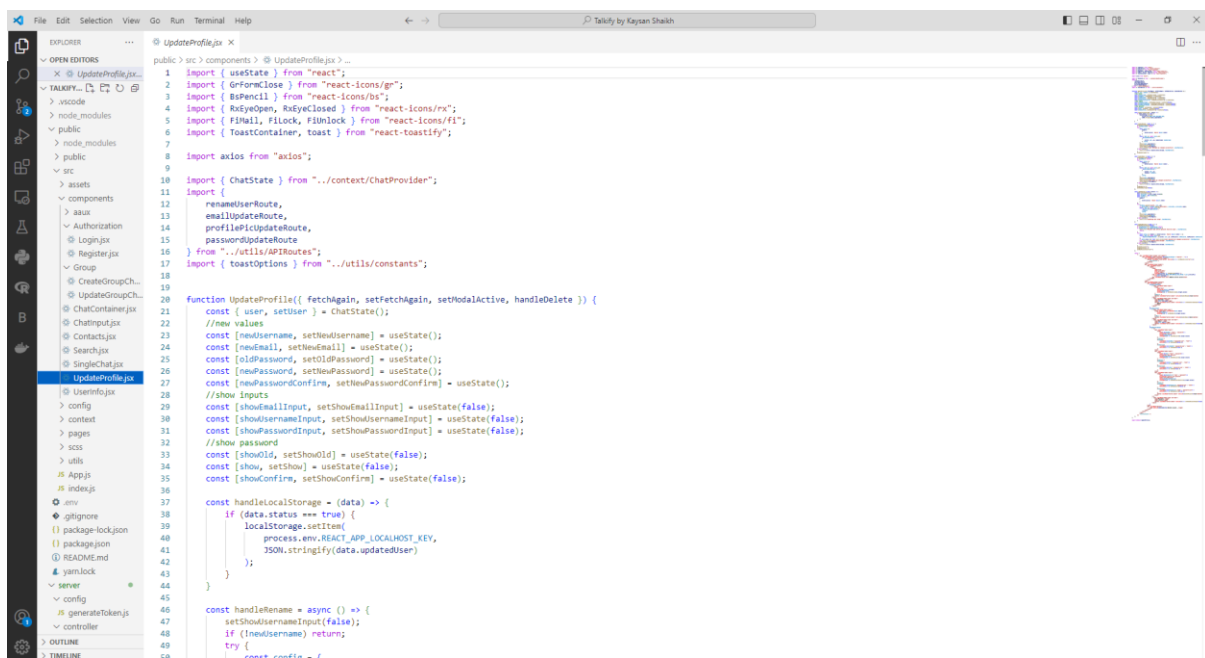
```




```

Search.jsx X
public > src > components > Search.jsx > ...
1 import "react-toastify/dist/ReactToastify.css";
2 import { ToastContainer, toast } from "react-toastify";
3 import axios from "axios";
4 import { accessChatRoute } from "../utils/APIRoutes";
5 import { toastOptions } from "../utils/constants";
6 import { ChatState } from "../context/ChatProvider";
7 import UserListItem from "../aux/UserListItem";
8
9 function Search({ searchResults }) {
10   const {
11     setSelectedChat,
12     user,
13     chats,
14     setChats,
15   } = ChatState();
16
17   const accessChat = async (userId) => {
18     try {
19       const config = { headers: {
20         "Content-type": "application/json",
21         Authorization: `Bearer ${user.token}`
22       } };
23     };
24     const postUsers = async () => {
25       const { data } = await axios.post(`${accessChatRoute}`, { userId }, config);
26       if (!chats.find((chat) => chat._id === data._id)) setChats([data, ...chats]);
27       setSelectedChat(data);
28     };
29     postUsers();
30     toast.success("The user is added to your contacts");
31   } catch (error) { toast.error("Error fetching the chat", toastOptions); }
32 };
33
34 return (
35   <>
36     <div className="search">
37       {searchResults.length !== 0 ? (
38         <div className="contacts">
39           {searchResults.map((result) => (
40             <UserListItem handleFunction={() => accessChat(result._id)}
41               result={result}
42               key={result._id} />
43           ))}
44         </div>
45       ) : (
46         <div className="chat-loading">
47           <b>No users found</b>
48         </div>
49       )}
50     </div>

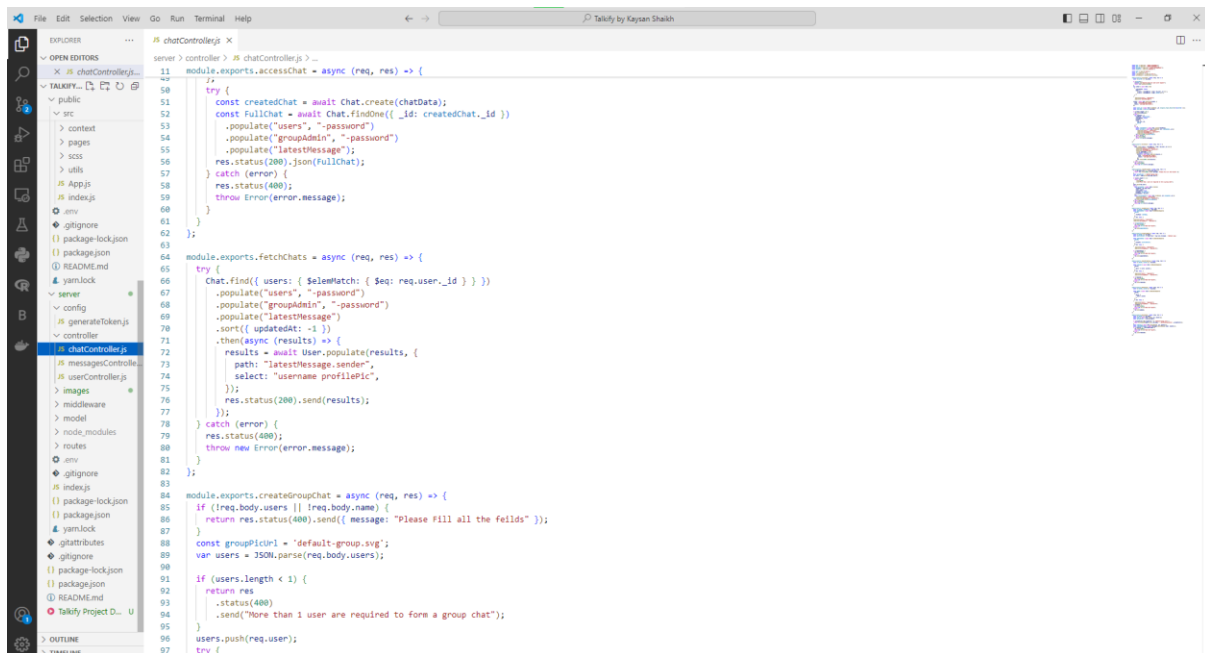
```



UserInfo.jsx

public > src > components > UserInfo > ...

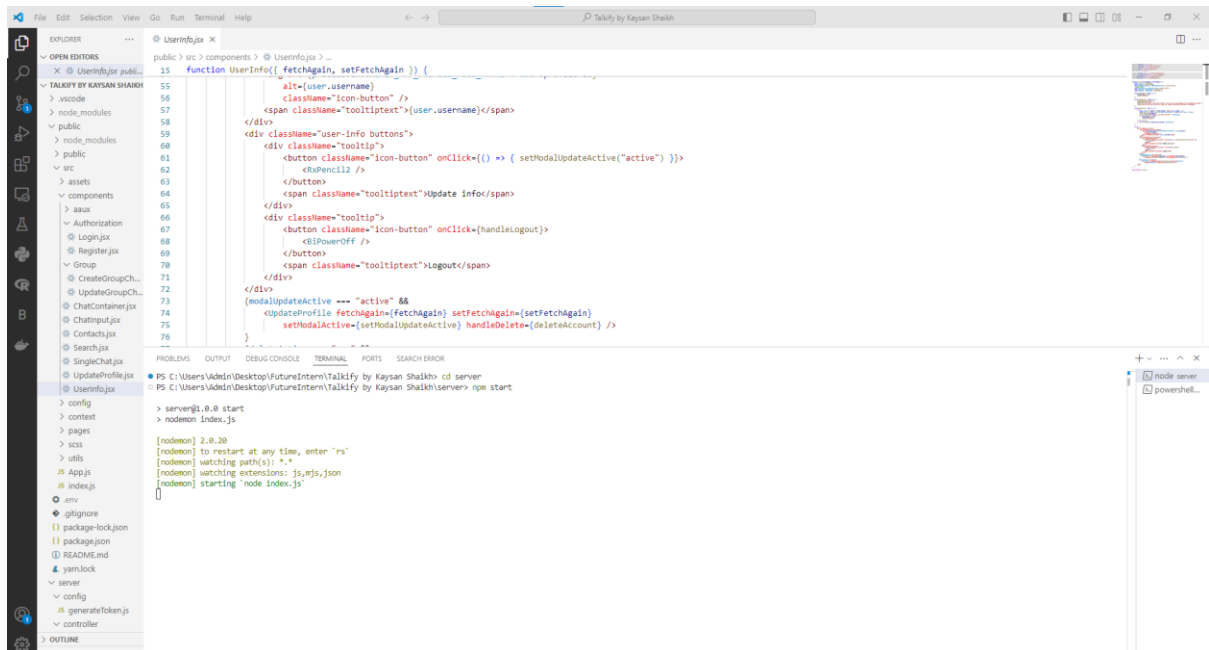
```
54     <img src={process.env.REACT_APP_PROFILE_PICS_PATHS + user.profilePic}
55         alt={user.username}
56         className="icon-button" />
57     <span className="tooltiptext">{user.username}</span>
58 </div>
59 <div className="user-info buttons">
60     <div className="tooltip">
61         <button className="icon-button" onClick={() => { setModalUpdateActive("active") }}>
62             <RxPencil2 />
63         </button>
64         <span className="tooltiptext">Update info</span>
65     </div>
66     <div className="tooltip">
67         <button className="icon-button" onClick={handleLogout}>
68             <BiPowerOff />
69         </button>
70         <span className="tooltiptext">Logout</span>
71     </div>
72 </div>
73 {modalUpdateActive === "active" &&
74     <UpdateProfile fetchAgain={fetchAgain} setFetchAgain={setFetchAgain}
75     setModalActive={setModalUpdateActive} handleDelete={deleteAccount} />
76 }
77 {deleteActive === "yes" &&
78     <SubmitModal setModalActive={setDeleteActive} warnText={warnText}
79     submText={submText} handleFunction={handleDelete} />
80 }
81 </div>
82 );
83 }
84
85 export default UserInfo;
```



```
server > controller > JS messagesController.js > ...
1 const Messages = require("../model/messageModel");
2 const User = require("../model/userModel");
3 const Chat = require("../model/chatModel");
4
5 module.exports.getMessages = async (req, res, next) => {
6   try {
7     const messages = await Messages.find({ chat: req.params.chatId })
8     .populate("sender", "username profilePic email")
9     .populate({ path: "chat", populate: { path: "latestMessage", populate: { path: "sender" } } });
10    res.json(messages);
11  } catch (error) {
12    res.status(400);
13    throw new Error(error.message);
14  }
15 };
16
17 module.exports.sendMessage = async (req, res, next) => {
18   const { sender, chatId, content } = req.body;
19   const attachmentUrl = (req.file ? req.file.filename : '');
20
21   if (!chatId) {
22     console.log("Invalid data passed into request");
23     return res.sendStatus(400);
24   }
25
26   var newMessage = {
27     sender: sender,
28     content: content,
29     chat: chatId,
30     attachment: attachmentUrl,
31   };
32
33   var message = await Messages.create(newMessage);
34   await Chat.findByIdAndUpdate(chatId, {
35     latestMessage: message
36   }, {
37     new: true,
38   });
39
40   message = await message.populate("sender", "username profilePic");
41   message = await User.populate(message, {
42     path: "chat.users",
43     select: "username profilePic",
44   });
45   message = await message.populate({ path: "chat", populate: { path: "latestMessage", populate: { path: "sender" } } });
46   res.json(message);
47 } catch (error) {
48   res.status(400);
49   next(error);
50 }
```

```
server > controller > JS userController.js > register > hashedPassword
1 const User = require("../model/userModel");
2 const Chat = require("../model/chatModel");
3 const Messages = require("../model/messageModel");
4 const bcrypt = require("bcrypt");
5 const config = require("../config");
6 const path = require("path");
7 const fs = require("fs");
8 const { promisify } = require("util");
9 const unlinkAsync = promisify(fs.unlink);
10 const { generateToken } = require("../config/generateToken");
11
12 module.exports.register = async (req, res, next) => {
13   try {
14     const { username, email, password } = req.body;
15     const usernameCheck = await User.findOne({ username });
16     const emailCheck = await User.findOne({ email });
17     const profilePicUrl = (req.file ? req.file.filename : 'default.svg');
18
19     if (usernameCheck) {
20       return res.json({ msg: "The username is already used", status: false });
21     }
22     if (emailCheck) {
23       return res.json({ msg: "The email is already used", status: false });
24     }
25     const hashedPassword = await bcrypt.hash(password, 10);
26     const user = await User.create({
27       username,
28       email,
29       password: hashedPassword,
30       profilePic: profilePicUrl,
31     });
32     delete user.password;
33     return res.json({
34       status: true, user: {
35         _id: user._id,
36         username: user.username,
37         email: user.email,
38         profilePic: user.profilePic,
39         isAdmin: user.isAdmin,
40         token: generateToken(user._id),
41       }
42     });
43   } catch (e) {
44     next(e);
45   }
46 };
47
48 module.exports.login = async (req, res, next) => {
49   try {
50     const { username, password } = req.body;
```

Server:



```
15 function Userinfo({ fetchAgain, setFetchAgain }) {
55   alt={user.username}
56   className="icon-button" />
57   <span className="tooltip">{user.username}</span>
58 </div>
59 <div className="user-info buttons">
60   <div className="tooltip">
61     <button className="icon-button" onClick={() => { setModalUpdateActive("active") }}>
62       <img alt="pencil icon" />
63     </button>
64     <span className="tooltip">Update info</span>
65   </div>
66   <div className="tooltip">
67     <button className="icon-button" onClick={handleLogout}>
68       <img alt="power off icon" />
69     </button>
70     <span className="tooltip">Logout</span>
71   </div>
72 </div>
73 (modalUpdateActive === "active" &&
74   <updateProfile fetchAgain={fetchAgain} setFetchAgain={setFetchAgain}
75     setModalActive={setModalUpdateActive} handleDelete={deleteAccount} />
76 )

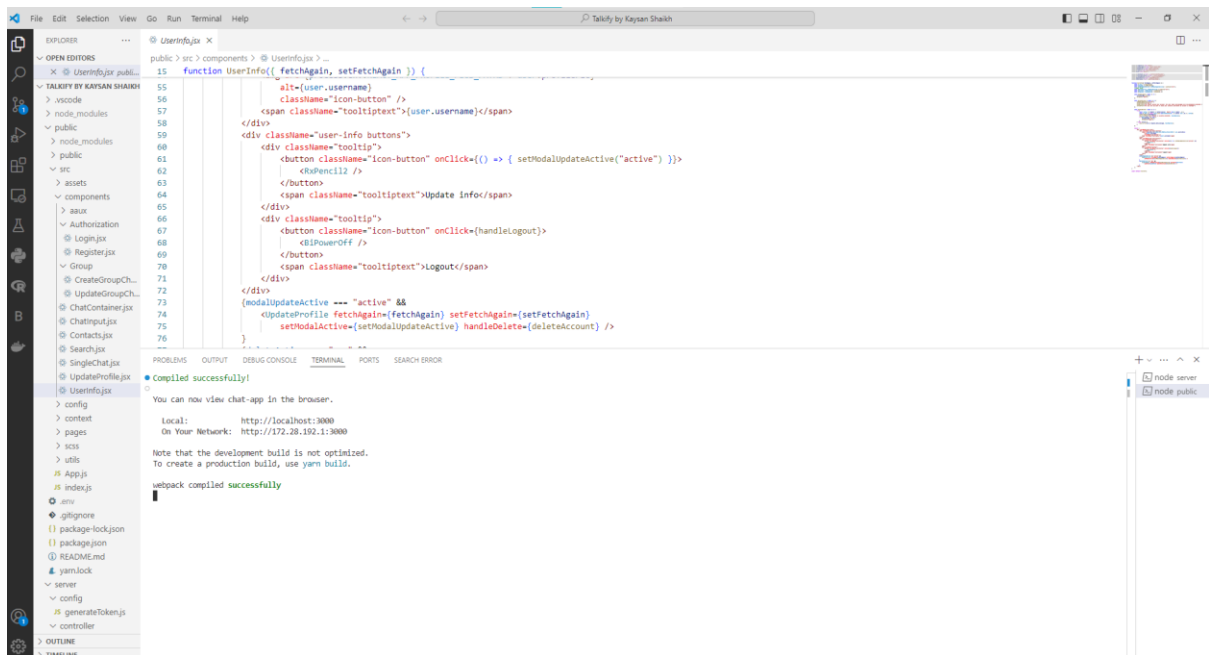
```

PS C:\Users\Adin\Desktop\FutureIntern\Talkify by Kayvan Shaikh> cd server
PS C:\Users\Adin\Desktop\FutureIntern\Talkify by Kayvan Shaikh> npm start

server@1.0.0 start
nodemon index.js

[nodemon] 2.0.20
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,json
[nodemon] starting 'node index.js'

Client:



```
15 function Userinfo({ fetchAgain, setFetchAgain }) {
55   alt={user.username}
56   className="icon-button" />
57   <span className="tooltip">{user.username}</span>
58 </div>
59 <div className="user-info buttons">
60   <div className="tooltip">
61     <button className="icon-button" onClick={() => { setModalUpdateActive("active") }}>
62       <img alt="pencil icon" />
63     </button>
64     <span className="tooltip">Update info</span>
65   </div>
66   <div className="tooltip">
67     <button className="icon-button" onClick={handleLogout}>
68       <img alt="power off icon" />
69     </button>
70     <span className="tooltip">Logout</span>
71   </div>
72 </div>
73 (modalUpdateActive === "active" &&
74   <updateProfile fetchAgain={fetchAgain} setFetchAgain={setFetchAgain}
75     setModalActive={setModalUpdateActive} handleDelete={deleteAccount} />
76 )

```

Compiled successfully!

You can now view chat-app in the browser.

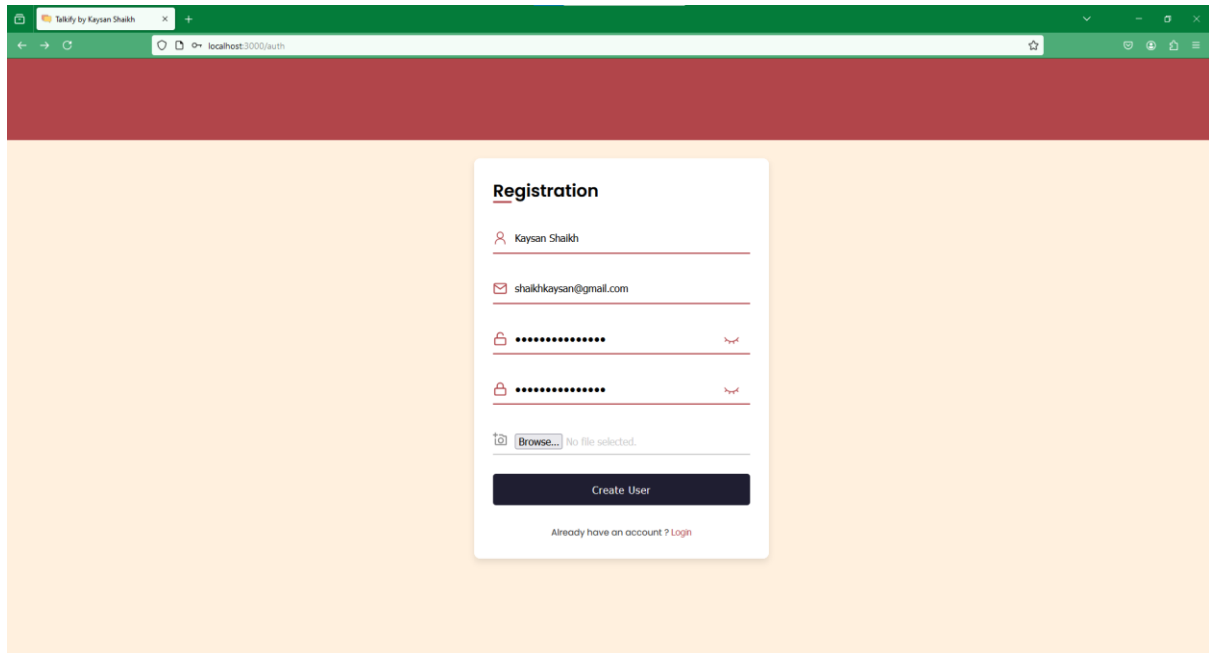
Local: http://localhost:3000
On Your Network: http://172.28.192.1:3000

Note that the development build is not optimized.
To create a production build, use `npm run build`.

webpack compiled successfully

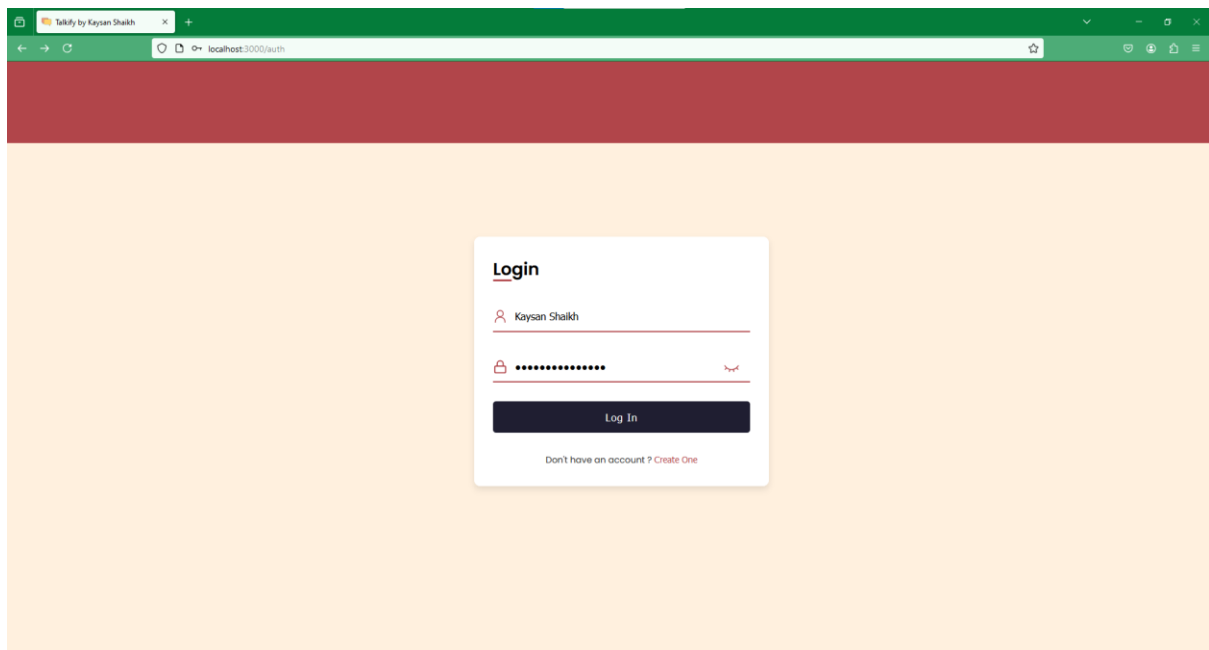
Output: -

Creating a user account i.e my account



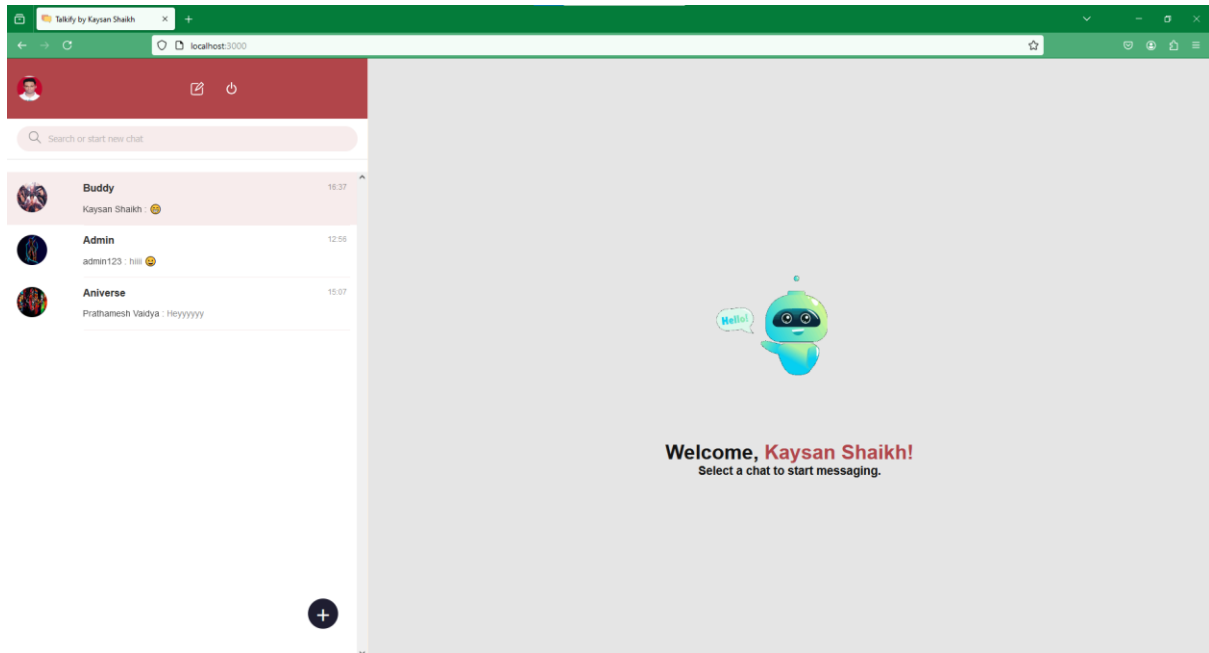
The screenshot shows a web browser window with the title "Talkify by Kaysan Shaikh". The address bar displays "localhost:3000/auth". The page has a dark red header and a light orange background. A white registration form is centered on the page. The form is titled "Registration" and contains the following fields: a name field with the value "Kaysan Shaikh", an email field with the value "shaikhkaysan@gmail.com", two password fields (both masked with dots), and a file upload field with the text "Browse..." and "No file selected.". Below the fields is a dark blue button labeled "Create User". At the bottom of the form, there is a link that says "Already have an account ? Login".

Logging in

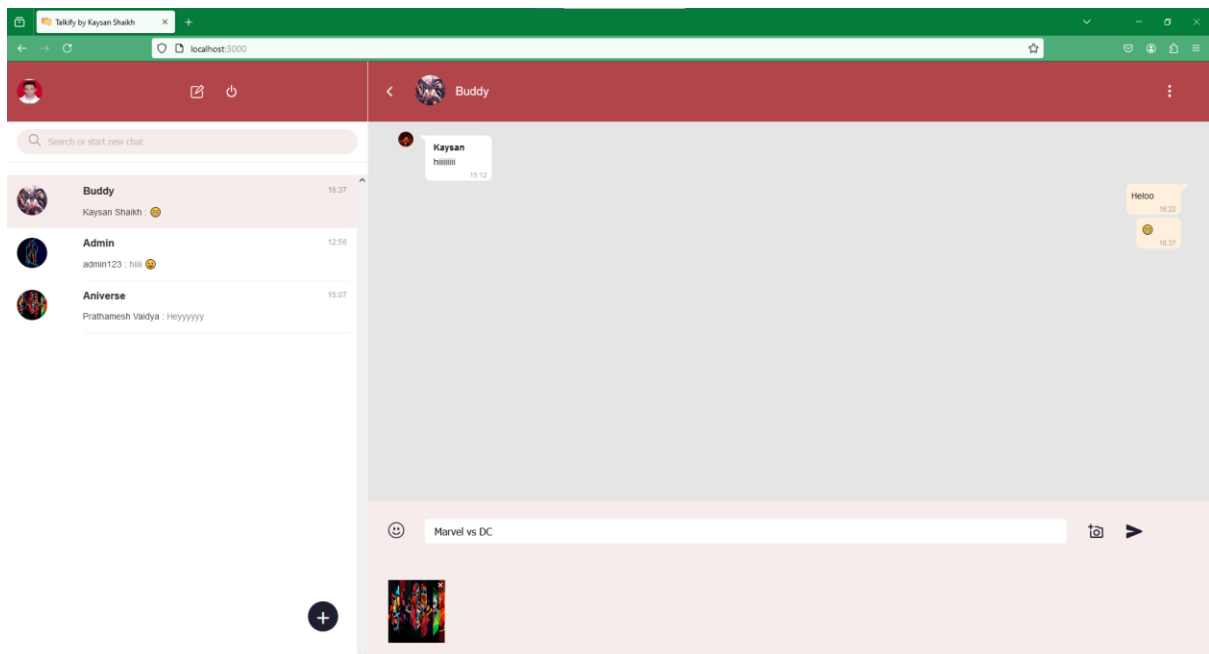


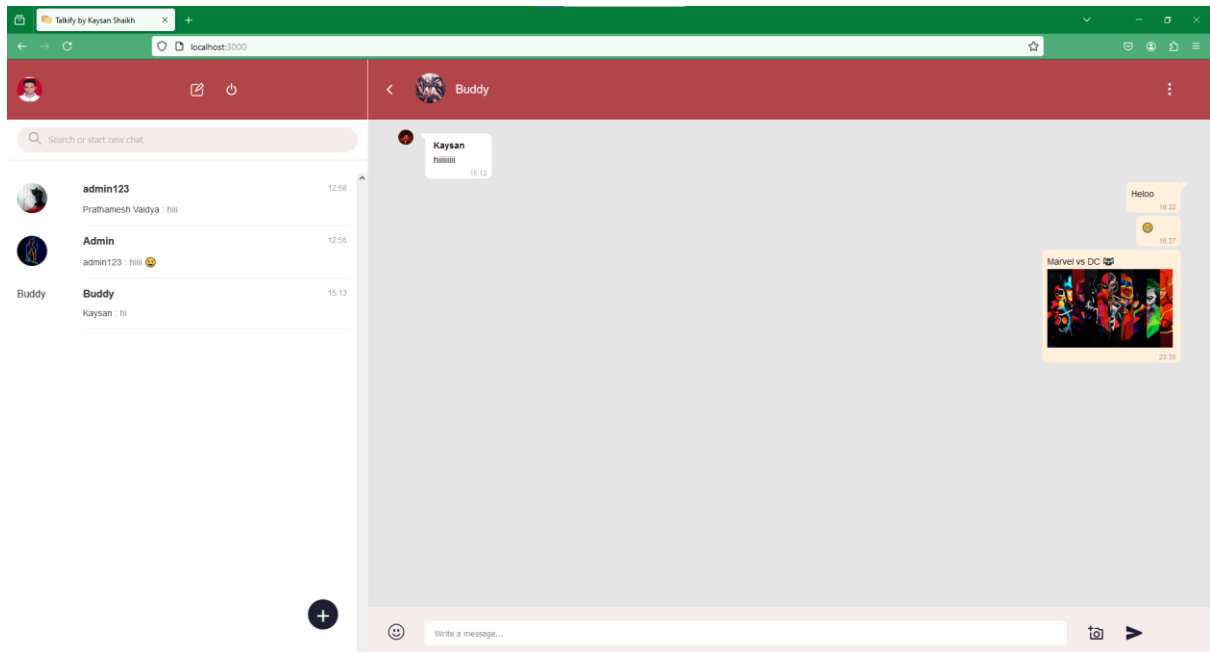
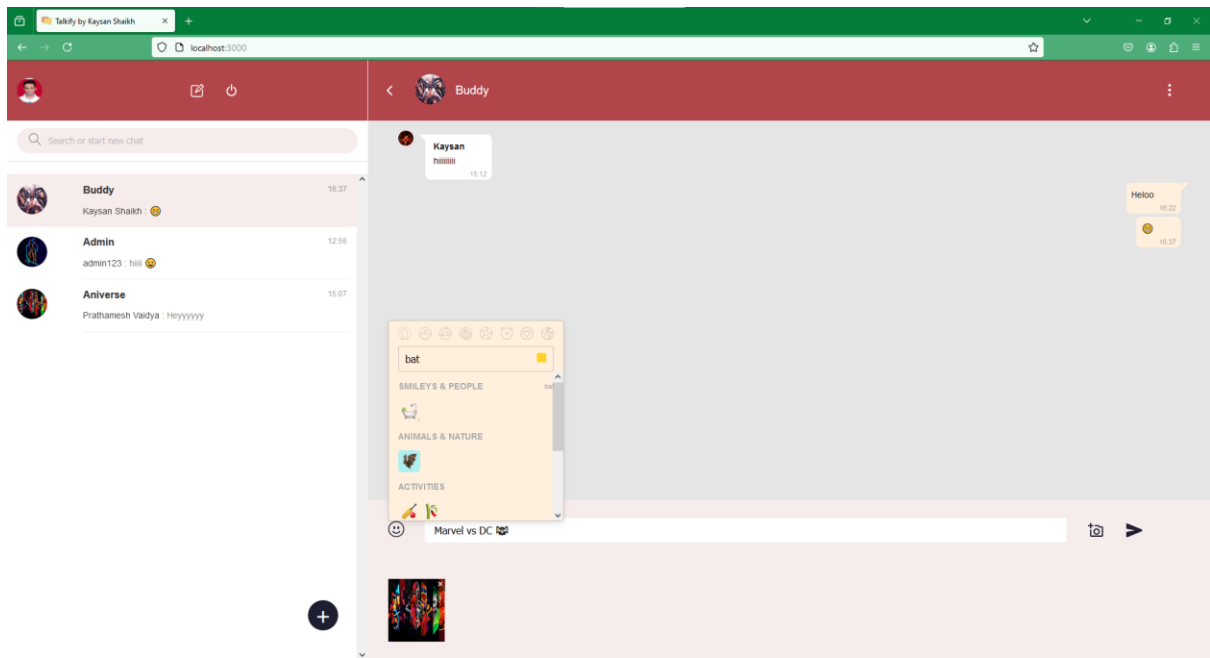
The screenshot shows a web browser window with the title "Talkify by Kaysan Shaikh". The address bar displays "localhost:3000/auth". The page has a dark red header and a light orange background. A white login form is centered on the page. The form is titled "Login" and contains the following fields: a name field with the value "Kaysan Shaikh" and a password field (masked with dots). Below the fields is a dark blue button labeled "Log In". At the bottom of the form, there is a link that says "Don't have an account ? Create One".

Logged in

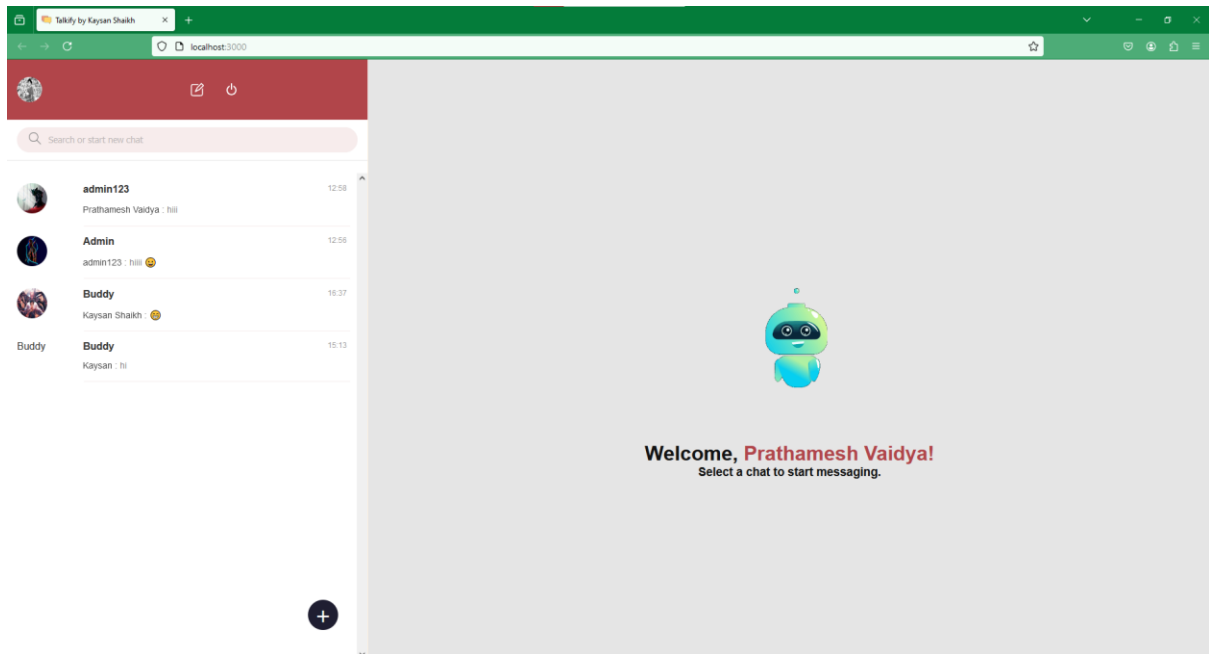


Sending a message in a group

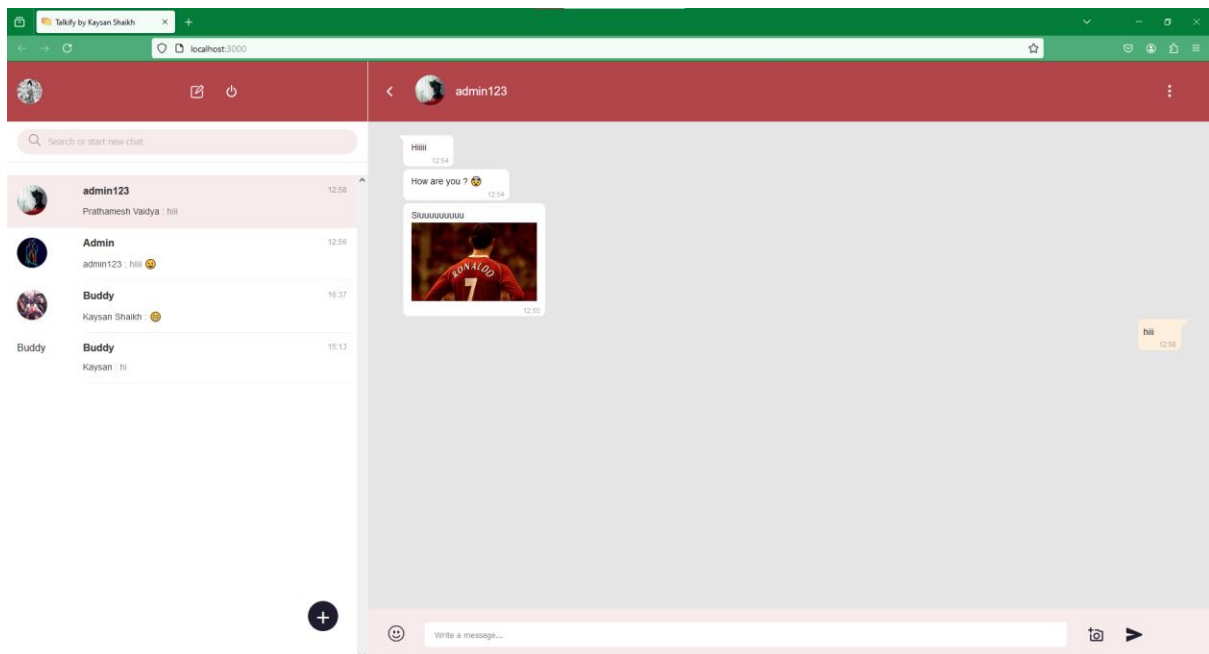


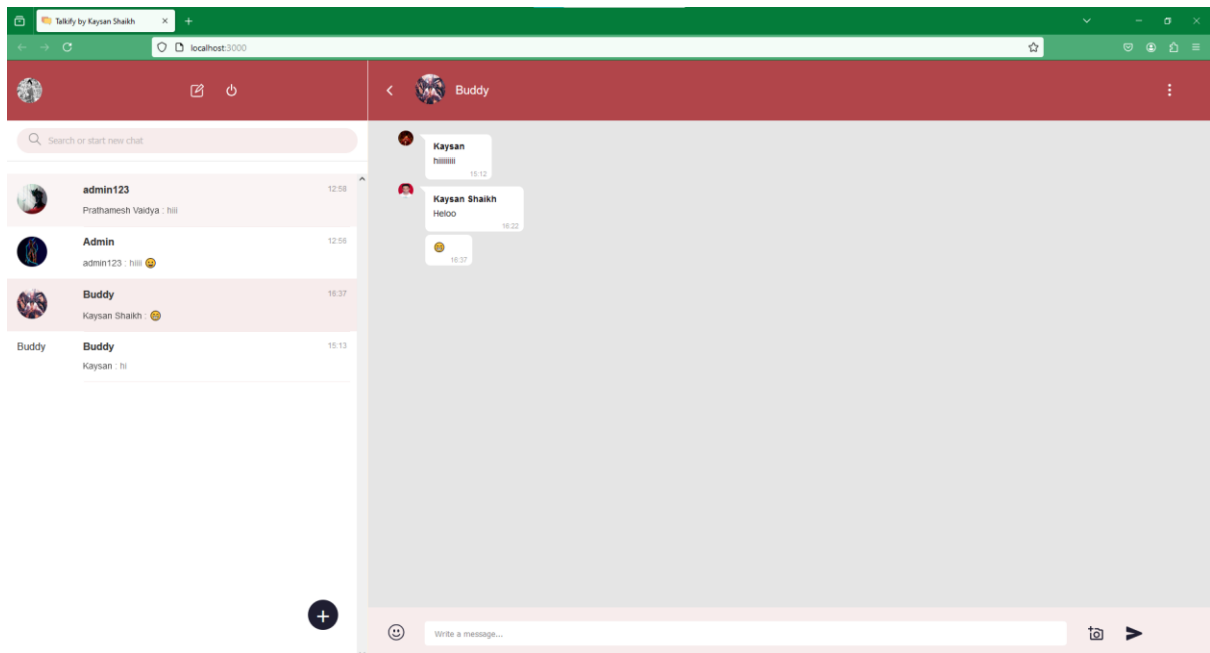


Logged in as a different user

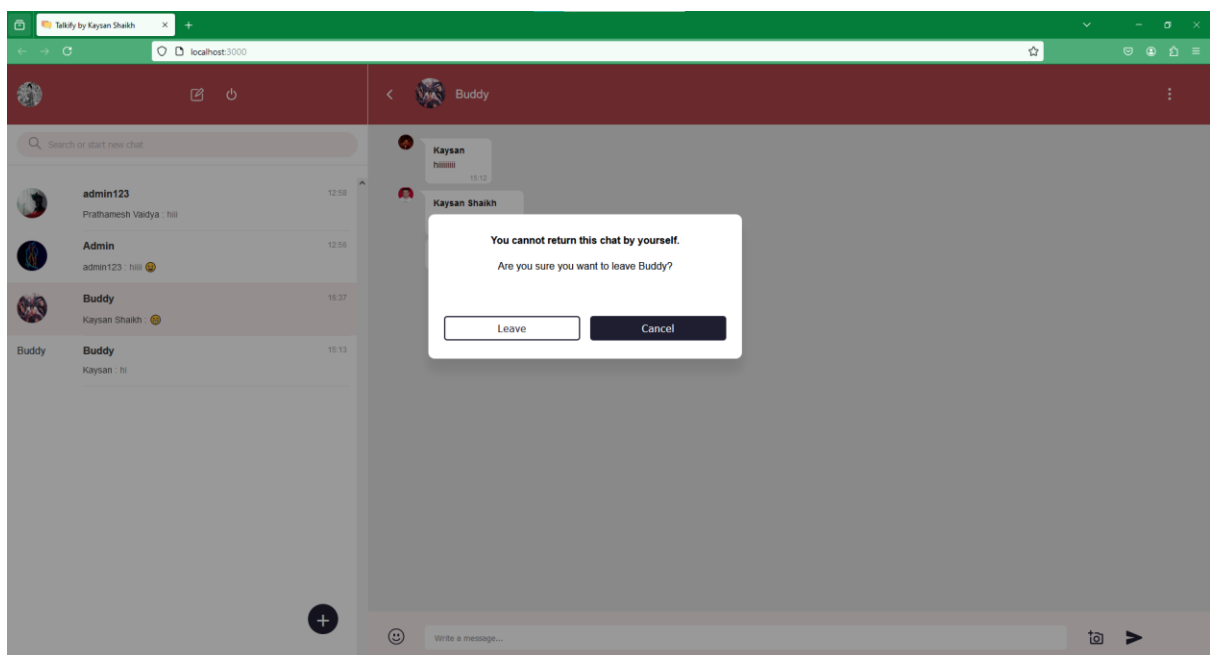


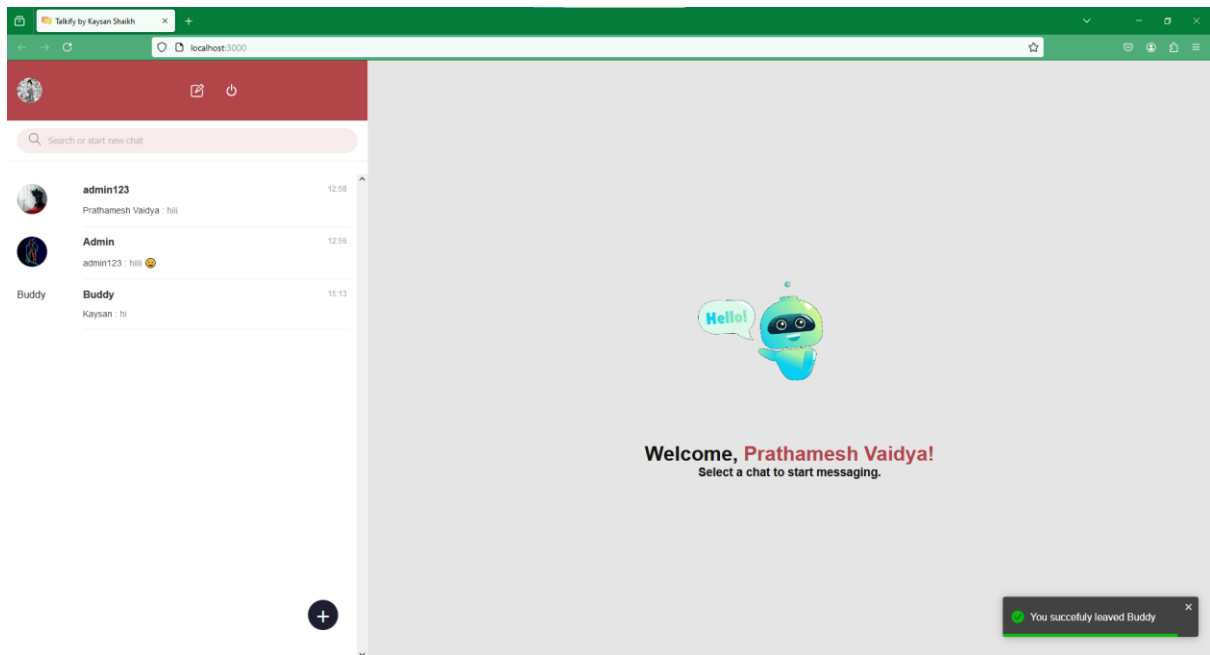
Sending a message to a user



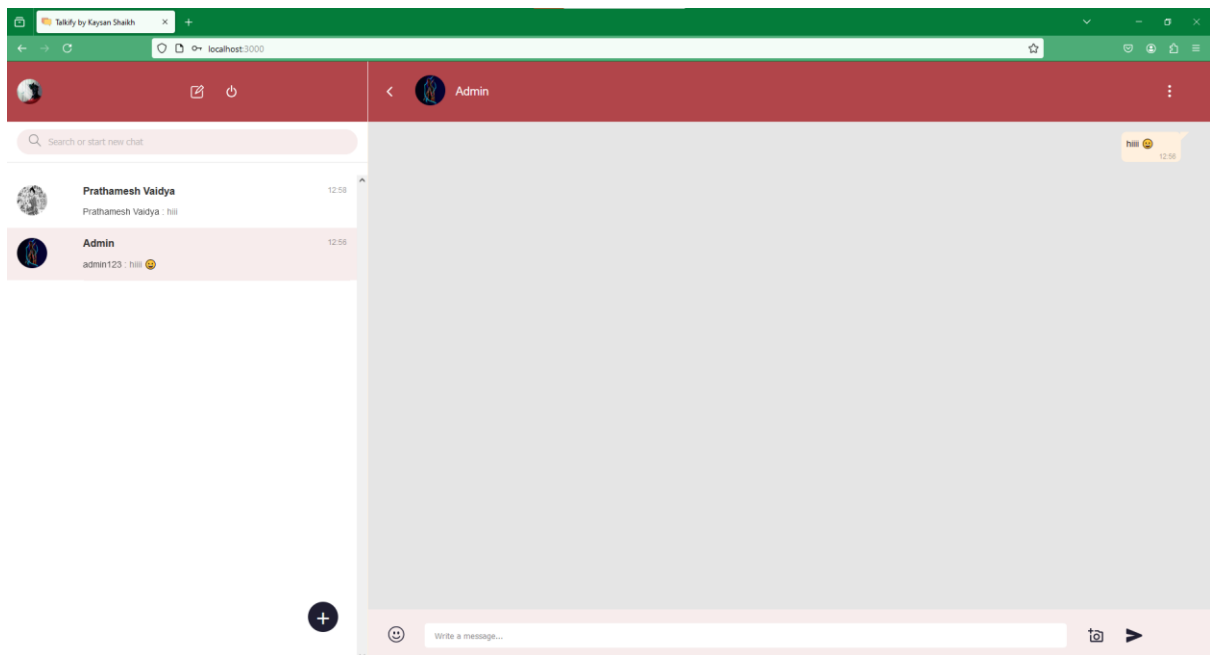


Leaving a group

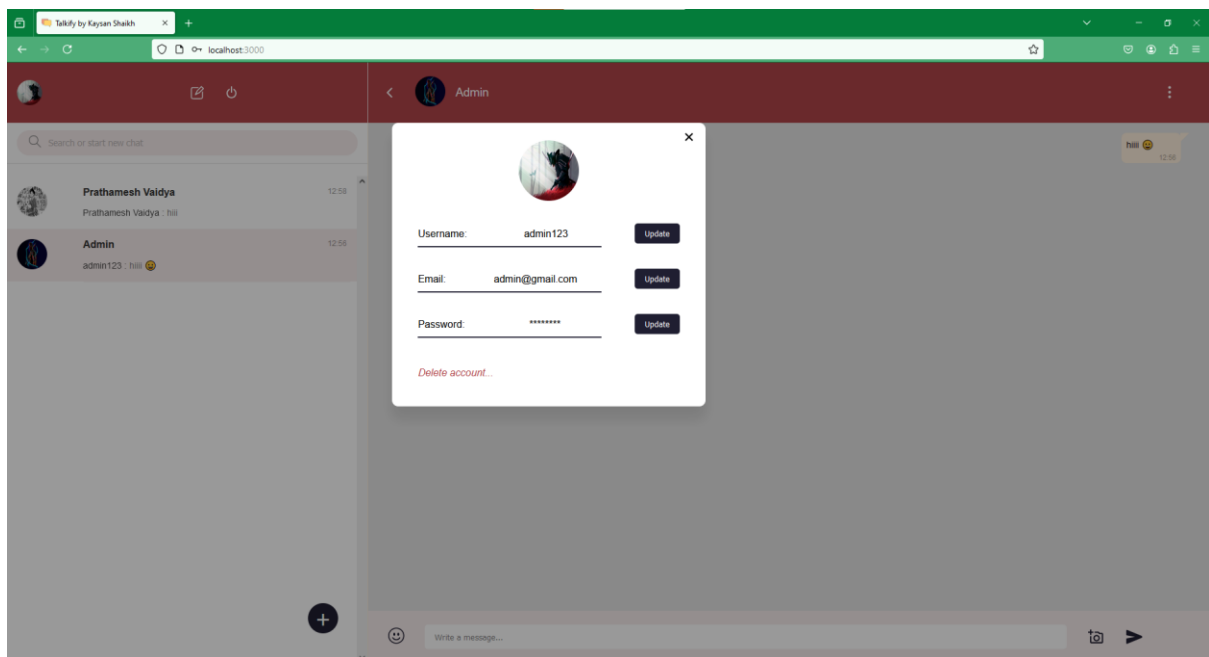




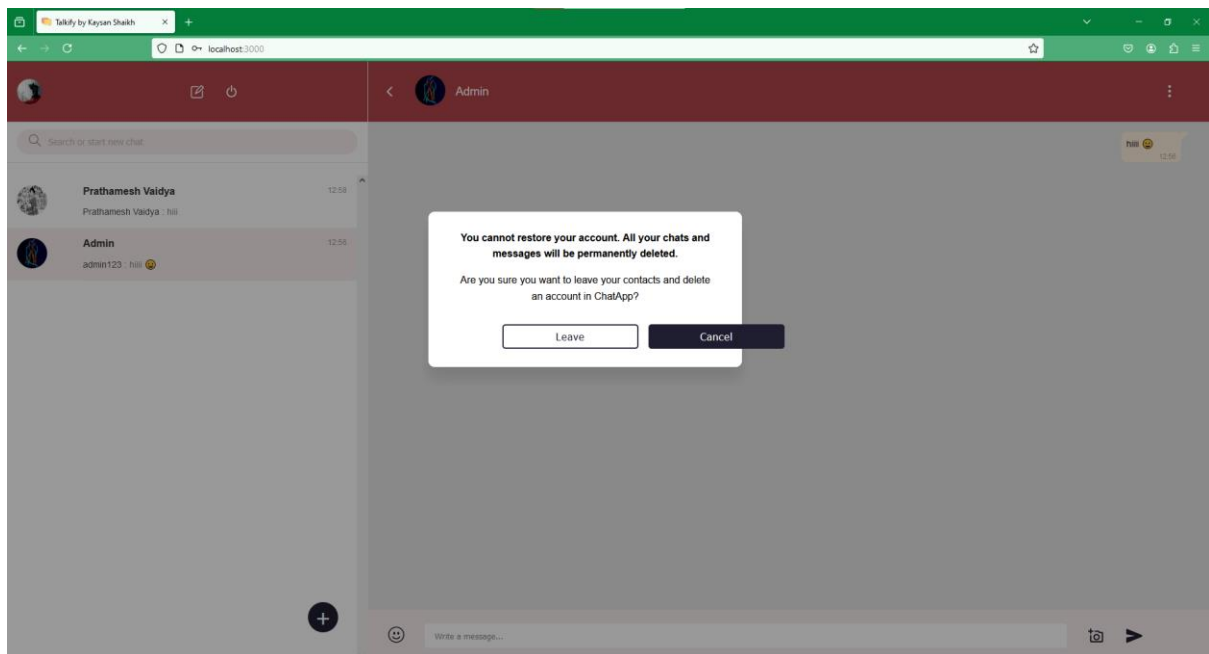
Logged in as a admin



Updating a user



Deleting a user



Conclusion: -

Talkify By Kaysan Shaikh is a comprehensive real-time chat application that showcases the power and flexibility of the MERN stack. By integrating real-time messaging, user authentication, group chats, message history, and media sharing, Talkify provides users with a seamless and interactive communication platform. The application's architecture, built using modern web technologies like MongoDB, Express.js, React, Node.js, and Socket.IO, ensures a scalable, secure, and responsive user experience.

Licensing: -

This project is licensed under the MIT License. This means that the source code is freely available for anyone to use, modify, and distribute. Users are welcome to contribute to the project or use the codebase as a foundation for their own projects, as long as proper credit is given.

Future Scopes: -

While Talkify already offers a robust set of features, there are several areas for future enhancement:

1. **Advanced User Management:** Introduce roles such as moderators or administrators to manage group chats, enforce rules, and maintain the quality of interactions.
2. **End-to-End Encryption:** Implement end-to-end encryption for all messages to enhance security and ensure that conversations remain private and secure.
3. **Push Notifications:** Integrate push notifications for real-time alerts, ensuring that users are notified of new messages, even when they are not actively using the application.
4. **Voice and Video Chat:** Expand the application's functionality to include voice and video chat capabilities, providing users with a more versatile communication platform.
5. **Mobile Application:** Develop a mobile version of Talkify to cater to users who prefer to chat on the go, offering the same seamless experience across different devices.
6. **Custom Themes:** Allow users to customize the appearance of the chat application by selecting from different themes or creating their own.